

Основы функционального программирования

Практическая работа №4 Монады и побочные эффекты

Цель: Освоить работу с монадами Maybe, Either, IO для обработки ошибок и ввода-вывода.

Теоретическая справка

- `Maybe a` — вычисление, которое может завершиться неудачей.
- `Either e a` — вычисление с детализированной ошибкой типа `e`.
- Монадические операторы:
 - `(>>=) :: m a -> (a -> m b) -> m b`
 - `return :: a -> m a`
- `do`-нотация — синтаксический сахар для связывания монадических действий.
- `IO a` — действие, которое при выполнении возвращает значение типа `a`.

Пример решения

Задача: Реализовать безопасное деление цепочки чисел: `[a,b,c,...] → a / b / c / ...`, возвращая `Either String Double`.

```
-- Безопасное деление двух чисел
safeDiv :: Double -> Double -> Either String Double
safeDiv _ 0 = Left "Division by zero"
safeDiv x y = Right (x / y)

-- Цепочка делений через свёртку с >>= (реализация через foldlM)
chainDiv :: [Double] -> Either String Double
chainDiv [] = Left "Empty list"
chainDiv [x] = Right x
chainDiv (x:y:ys) = foldl step (safeDiv x y) ys
  where
    step (Right acc) z = safeDiv acc z
    step (Left err) _ = Left err
```

```
-- Альтернатива через явное связывание
chainDiv' :: [Double] -> Either String Double
chainDiv' []      = Left "Empty list"
chainDiv' (x:xs) = go x xs
  where
    go acc []      = Right acc
    go acc (y:ys) = case safeDiv acc y of
      Left err -> Left err
      Right r  -> go r ys
```

Тестирование в GHCi

```
-- > chainDiv [100, 2, 5]
-- Right 10.0
-- > chainDiv [100, 0, 5]
-- Left "Division by zero"
-- > chainDiv []
-- Left "Empty list"
```

Ход работы

1. Создайте файл `Pr4_V<N>.hs`.
2. Для задач с обработкой ошибок используйте `Either String a`.
3. Для задач с вводом-выводом:
 - a. Все операции чтения/записи — внутри `IO`
 - b. Обработку ошибок файлов — через `catch` из `System.IO.Error`
4. Избегайте `unsafePerformIO` и `fromJust`.
5. Все потенциально падающие операции должны возвращать `Either` или `Maybe`
6. Избегать `fromJust`, `head`, `!!` без проверки
7. Протестируйте все сценарии: успешные и ошибочные.

Требования к отчёту

- Титульный лист (ФИО, группа, работа №1, вариант)
- Условие задачи
- Листинг кода с комментариями
- Схема потока данных в монадическом вычислении
- Примеры успешного выполнения и обработки ошибок
- Обоснование выбора `Maybe` vs `Either`
- Примеры использования всех функций
- Формат PDF

Варианты заданий

Вариант 1. Калькулятор выражений из файла

Компонент	Описание
Задача	Прочитать файл с арифметическими выражениями, вычислить каждое и вывести результаты или ошибки
Формат файла	Каждая строка — выражение: <code>2 + 3 * 4</code> Поддерживаемые операции: <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>
<code>parseExpr :: String -> Either String Expr</code>	Парсер выражения. Возвращает <code>Left "ошибка"</code> при синтаксической ошибке
<code>evalExpr :: Expr -> Either String Int</code>	Вычисление выражения. Возвращает <code>Left "Division by zero"</code> при делении на 0
<code>processFile :: FilePath -> IO ()</code>	Основная функция: 1. Читает файл по пути 2. Парсит каждую строку 3. Вычисляет корректные выражения 4. Выводит результаты в формате: <code>Line 1: 14</code> <code>Line 2: Error: Division by zero</code>
Граничные случаи	• Файл не найден → сообщение об ошибке в <code>IO</code> • Пустой файл → ничего не выводить • Пустая строка → пропустить или вывести <code>Error: Empty line</code>

Вариант 2. Загрузка конфигурации

Компонент	Описание
Задача	Загрузить конфигурацию из текстового файла в формате <code>ключ=значение</code>
Формат файла	Каждая строка: <code>имя=значение</code> Комментарии начинаются с <code>#</code> Пустые строки игнорируются
Тип <code>Config</code>	<code>type Config = [(String, String)]</code>
<code>loadConfig :: FilePath -> IO (Either String Config)</code>	Чтение файла и парсинг. Возвращает <code>Left</code> при ошибках: • Файл не найден • Некорректный формат строки (нет <code>=</code>)

	• Повторяющиеся ключи
<code>getOrDefault :: String -> String -> Config -> String</code>	Получить значение по ключу или вернуть значение по умолчанию
<code>getInt :: String -> Config -> Either String Int</code>	Безопасное получение целого числа
Граничные случаи	• Отсутствующий ключ → <code>getOrDefault</code> возвращает дефолтное значение • Невалидное целое → <code>Left "Parse error"</code> • Дублирование ключей → <code>Left "Duplicate key: host"</code>

Вариант 3. Интерактивный фильтр чисел

Компонент	Описание
Задача	Интерактивная программа: ввести список чисел, выбрать операцию, получить результат
<code>readIntList :: IO (Either String [Int])</code>	Чтение строки из консоли и парсинг в список целых чисел
<code>filterEven :: [Int] -> [Int]</code>	Фильтр чётных чисел
<code>filterOdd :: [Int] -> [Int]</code>	Фильтр нечётных чисел
<code>filterPrime :: [Int] -> [Int]</code>	Фильтр простых чисел
<code>main :: IO ()</code>	Интерактивный цикл: 1. Ввести числа (через пробел) 2. Выбрать операцию: 1=чётные, 2=нечётные, 3=простые, 0=выход 3. Вывести результат или ошибку
Граничные случаи	• Пустой ввод → <code>Left "Empty input"</code> • Некорректное число → сообщение об ошибке, повтор ввода • Неверный выбор операции → повтор выбора

Вариант 4. Чтение CSV-файла

Компонент	Описание
Задача	Прочитать CSV-файл (запятая как разделитель) в матрицу строк
<code>readCSV :: FilePath -> IO (Either</code>	Чтение и парсинг CSV. Обработка ошибок:

<code>String [[String]])</code>	• Файл не найден → <code>Left "File not found"</code> • Некорректный формат → <code>Left "Parse error at line N"</code>
<code>getColumn :: Int -> [[a]] -> Maybe [a]</code>	Получить столбец по индексу (0-based)
<code>getIntColumn :: Int -> [[String]] -> Either String [Int]</code>	Получить столбец и преобразовать в целые числа
Граничные случаи	• Пустой файл → <code>Right []</code> • Несоответствие количества столбцов → <code>Left "Inconsistent columns at line N"</code> • Пустые поля → <code>" "</code> как значение

Вариант 5. Валидация формы пользователя

Компонент	Описание
Задача	Валидация данных пользователя с детализированными сообщениями об ошибках
Тип <code>User</code>	<code>data User = User { name :: String, age :: Int, email :: String } deriving (Show)</code>
<code>validateName :: String -> Either String String</code>	Проверка имени: не пустое, минимум 2 символа
<code>validateAge :: String -> Either String Int</code>	Проверка возраста: целое число от 0 до 150
<code>validateEmail :: String -> Either String String</code>	Проверка email: содержит @ и точку после @
<code>validate :: String -> String -> String -> Either String User</code>	Комплексная валидация всех полей. Накапливает все ошибки через <code>do</code> -нотацию
Граничные случаи	• Все поля невалидны → все ошибки в одном сообщении • Частично валидны → только ошибки невалидных полей

Вариант 6. Цепочка безопасных преобразований

Компонент	Описание
Задача	Реализовать цепочку преобразований через монаду <code>Maybe</code> с использованием <code>(>>=)</code> и <code>do</code>

<code>parseInt :: String -> Maybe Int</code>	Парсинг строки в целое число
<code>toDouble :: Int -> Maybe Double</code>	Преобразование <code>Int</code> в <code>Double</code> (всегда успешно)
<code>multiplyByPi :: Double -> Maybe Double</code>	Умножение на π
<code>roundToTwoDecimals :: Double -> Maybe String</code>	Округление до 2 знаков и преобразование в строку
<code>chainWithBind :: String -> Maybe String</code>	Цепочка через <code>(>>=)</code>
<code>chainWithDo :: String -> Maybe String</code>	Та же цепочка через <code>do</code> -нотацию
Граничные случаи	• Некорректный ввод на любом этапе → <code>Nothing</code> • Отрицательные числа → обрабатываются корректно

Вариант 7. Игра «Угадай число»

Компонент	Описание
Задача	Компьютер загадывает число от 1 до 100, игрок угадывает с подсказками
<code>main :: IO ()</code>	Основной игровой цикл: 1. Сгенерировать случайное число (<code>randomRIO (1, 100)</code>) 2. Запросить ввод у игрока 3. Сравнить и дать подсказку ("больше"/"меньше"/"угадал") 4. Считать количество попыток 5. Спросить, хочет ли игрок сыграть ещё
<code>readGuess :: IO (Either String Int)</code>	Безопасное чтение числа с валидацией диапазона
Граничные случаи	• Некорректный ввод → повтор запроса с сообщением об ошибке • Число вне диапазона → сообщение и повтор запроса • EOF (Ctrl+D) → корректное завершение

Вариант 8. Безопасный поиск в ассоциативном списке

Компонент	Описание
Задача	Обёртка над <code>lookup</code> с детализированными сообщениями об ошибках

<code>safeLookup :: Eq k => k -> [(k,v)] -> Either String v</code>	Безопасный поиск. Возвращает <code>Left "Key not found: <key>"</code> при отсутствии ключа
<code>lookupChain :: [(String, String)] -> [String] -> Either String String</code>	Цепочка поисков: ищет первый ключ, если не найден — второй и т.д., накапливая путь ошибки
<code>lookupWithDefault :: Eq k => k -> v -> [(k,v)] -> v</code>	Поиск с значением по умолчанию (через <code>either</code>)
Граничные случаи	• Пустой список → <code>Left "Key not found"</code> • Несколько одинаковых ключей → возвращается первый

Вариант 9. Чтение нескольких файлов

Компонент	Описание
Задача	Прочитать содержимое нескольких файлов и объединить строки
<code>readFileSafe :: FilePath -> IO (Either String String)</code>	Безопасное чтение файла. Возвращает <code>Left</code> при ошибке
<code>readFilesSequential :: [FilePath] -> IO (Either String [String])</code>	Чтение списка файлов последовательно. Прерывается при первой ошибке
<code>concatenateFiles :: [FilePath] -> IO (Either String String)</code>	Объединение содержимого всех файлов в одну строку
Граничные случаи	• Пустой список файлов → <code>Right []</code> • Ошибка чтения любого файла → немедленное прерывание с сообщением

Вариант 10. Парсер скобочных выражений

Компонент	Описание
Задача	Проверка корректности скобочной последовательности с указанием позиции ошибки
<code>checkBrackets :: String -> Either (Int, String) ()</code>	Проверка скобок <code>()[]{}.</code> Возвращает <code>Right ()</code> при успехе или <code>Left (позиция, сообщение)</code> при ошибке
<code>main :: IO ()</code>	Интерактивный ввод строки и вывод результата проверки
Граничные случаи	• Пустая строка → <code>Right ()</code> • Только открывающие скобки → ошибка "незакрытая скобка"

	• Только закрывающие скобки → ошибка "неожиданная закрывающая скобка"
--	---

Вариант 11. Ввод матрицы из консоли

Компонент	Описание
Задача	Интерактивный ввод матрицы целых чисел с валидацией
<code>readMatrix :: IO (Either String (Matrix Int))</code>	Чтение матрицы: 1. Запросить количество строк и столбцов 2. Прочитать каждую строку (числа через пробел) 3. Проверить количество элементов в каждой строке
Тип <code>Matrix a</code>	<code>data Matrix a = Matrix { rows :: Int, cols :: Int, data :: [[a]] }</code>
Граничные случаи	• Некорректное число строк/столбцов → <code>Left "Invalid dimensions"</code> • Несоответствие количества элементов → <code>Left "Row 2: expected 3 elements, got 2"</code> • Некорректное число в строке → <code>Left "Invalid number at row 1, col 2"</code>

Вариант 12. Цепочка деления

Компонент	Описание
Задача	Последовательное деление чисел с обработкой деления на ноль через <code>Maybe</code>
<code>safeDiv :: Double -> Double -> Maybe Double</code>	Безопасное деление
<code>divideChain :: [Double] -> Maybe Double</code>	Последовательное деление: <code>x₁ / x₂ / x₃ / ...</code>
<code>divideChainFoldM :: [Double] -> Maybe Double</code>	Реализация через <code>foldM</code> из <code>Control.Monad</code>
Граничные случаи	• Пустой список → <code>Nothing</code> • Один элемент → <code>Just x</code> • Деление на ноль на любом шаге → <code>Nothing</code>

Вариант 13. Логирование вычислений

Компонент	Описание
Задача	Создать монаду для вычислений с логированием
Тип <code>Log a</code>	<code>data Log a = Log { logMessages :: [String], logResult :: a }</code>
<code>instance Monad Log</code>	Реализация <code>return</code> и <code>(>>=)</code> для накопления логов
<code>logMessage :: String -> Log ()</code>	Добавление сообщения в лог
<code>factorialWithLog :: Integer -> Log Integer</code>	Факториал с логированием каждого шага
<code>runLog :: Log a -> (a, [String])</code>	Извлечение результата и лога
Граничные случаи	• Пустой лог → <code>[]</code> • Вложенные вычисления → логи конкатенируются

Вариант 14. Суммирование чисел из файла с пропуском ошибок

Компонент	Описание
Задача	Прочитать файл с числами, просуммировать корректные, подсчитать ошибки
<code>parseAndSum :: FilePath -> IO (Double, Int)</code>	Чтение файла и обработка: • Возвращает пару: (сумма корректных чисел, количество пропущенных строк)
Граничные случаи	• Пустой файл → <code>(0.0, 0)</code> • Все строки некорректны → <code>(0.0, N)</code> • Файл не найден → обработка через <code>catch</code>

Вариант 15. Интерактивное меню калькулятора

Компонент	Описание
Задача	Интерактивное меню с операциями над двумя числами
<code>main :: IO ()</code>	Цикл меню: 1. Вывести меню: <code>1) Add 2) Multiply 3) Exit</code> 2. Прочитать выбор

	3. Если 1 или 2 — запросить два числа и вывести результат 4. Если 3 — выйти 5. При ошибке ввода — повторить
<code>readNumber :: IO (Either String Double)</code>	Безопасное чтение числа
Граничные случаи	• Некорректный выбор меню → повтор запроса • Некорректное число → повтор запроса числа с сообщением об ошибке